

Zero-Shot Adaptation for Guided Robot Policies^a

Dhruv Patel, Kushal Shah, Woo Chul Shin & Aryan Sarswat
Georgia Institute of Technology
{dpate1756, kshah422, wshin49, asarswat8}@gatech.edu

December 9, 2024

Abstract: This study introduces a novel framework for zero-shot adaptation of robot policies in real-time using a combination of diffusion-based trajectory planners and large language models (LLMs). Unlike traditional approaches that require retraining for new tasks, our system dynamically generates task-specific loss functions via LLMs to guide trajectory adaptation. The approach is tested on tasks involving speed and spatial constraints in the MetaWorld environment, demonstrating significant improvements in task success rates under unseen conditions. This work highlights the potential for flexible and efficient robot control in dynamic and collaborative environments, paving the way for more adaptable robotic systems.

Keywords: Diffusion Model, LLMs, Trajectory Planning, Loss guidance

1 Introduction

The demand for adaptable and versatile robots in dynamic environments is growing, particularly in human-robot collaboration scenarios like assembly lines, where task requirements and contexts frequently change. Traditional robotic systems, relying on task-specific training, face limitations due to their rigidity and inability to handle unforeseen scenarios without extensive retraining. This project addresses the core challenge of enabling robots to autonomously adapt their behaviors to user-defined contexts without prior exposure. By developing a language-conditioned robot policy, we aim to allow robots to dynamically adjust their actions based on high-level instructions and environmental cues, achieving zero-shot policy adaptation and seamless task execution.

Our approach combines diffusion-based trajectory planners with large language models (LLMs) to enable real-time behavioral adjustments. The diffusion model refines action trajectories, while the LLM generates task-specific loss functions based on real-time user inputs, such as spatial boundaries or execution speed. This system eliminates the need for pre-trained policies and extensive retraining, allowing robots to adapt their actions on demand. By testing this method in various scenarios and benchmarking its performance against established Deep RL standards, we aim to demonstrate its potential for flexible and robust robot behavior in real-world collaborative tasks.

2 Related Work

One of the biggest challenges in robotics is real-time behavior adaptation to changing needs, especially for activities that call for flexibility and responsiveness. For real-time applications, traditional methods frequently require retraining models to account for novel events, which is impracticable. Recent developments in large language models (LLMs) and diffusion models have shown encouraging potential for behavior adaption and trajectory generation. In order to overcome the shortcomings in multimodal task generalization and real-time adaptability, our work expands and develops upon these approaches, integrating their advantages.

2.1 Diffusion Models in Robotic Planning

Diffusion models, initially designed for generative modeling [3], have gained popularity in robotic planning due to their ability to produce intricate trajectories. The Diffuser model [5] combines model-based reinforcement learning with a reward predictor J_ϕ , guiding the sampling process through reverse diffusion transitions to estimate cumulative rewards. While effective in dynamic tasks and unstructured environments, Diffuser’s adaptability to new user-defined scenarios is limited, as modifying it requires creating and training separate reward functions for each situation. Our approach addresses this limitation by dynamically generating context-specific loss functions in real-time, enabling adaptation without retraining.

Similarly, the Diffusion Policy method [2] applies diffusion models to visuomotor policy learning, showing strong performance in simulated and real-world settings. However, it shares the retraining limitation for new environments. In contrast, our method integrates LLMs to generate task-specific loss functions on demand, guiding trajectory adaptation for zero-shot policy adaptation, overcoming the constraints of existing diffusion-based approaches.

2.2 LLM in Robotics

Large language models (LLMs) are increasingly used in robotics for tasks like creating reward functions and expressing behaviors through natural language. EUREKA [6], for instance, uses LLMs to automatically generate reward functions from environment code, outperforming human-designed rewards in many tasks after iterative improvements. However, EUREKA requires training from scratch, limiting its adaptability for real-time applications.

Our approach differs by focusing on generating loss functions post-training to directly influence trajectory adaptation in real-time. Using LLMs to dynamically create loss functions based on user-defined contexts enables zero-shot policy adaptation, eliminating the need for retraining and allowing immediate task adjustments. While other research explores LLMs for high-level planning like strategy formulation [1, 4], our work emphasizes real-time, lower-level trajectory adaptation, enabling robots to modify behaviors on the fly during task execution.

2.3 Our Approach

While previous work in diffusion models and LLMs has made significant strides in improving trajectory planning and model adaptation, these approaches generally fall short in enabling real-time, context-specific adaptation without retraining. Our approach addresses this gap by combining diffusion-based trajectory planners with LLMs to generate loss functions that guide trajectory adaptation in real-time. By conditioning the trajectory generation process on user-defined contexts—such as task speed, spatial constraints, or other behavioral requirements—we can adapt robot behaviors on the fly, without the need for retraining or task-specific preconditions.

Furthermore, by treating the generation of loss functions as a task-specific optimization problem, our system enables zero-shot policy adaptation at runtime, ensuring flexibility in dynamic environments. This approach not only builds on the strengths of diffusion models and LLMs but also presents a new paradigm for real-time policy adaptation that can be applied to a wide range of robotic applications, from collaborative assembly lines to autonomous navigation tasks.

3 Methods

Our proposed methodology integrates diffusion-based trajectory planners with LLMs to enable real-time, context-specific robot behavior adaptation. The approach is structured into three key stages: (1) training an initial diffusion model on a foundational task, (2) generating task-specific loss functions using an LLM, and (3) employing loss-guided diffusion to optimize trajectories for new tasks.

3.1 Training the Initial Diffusion model

The first stage involves training a diffusion model on an initial task, such as pick and place (without a wall). The diffusion model is trained as a conditional denoising process, where it learns to generate feasible trajectories by iteratively refining noisy inputs. The training objective minimizes the discrepancy between the generated trajectories and ground-truth trajectories from the dataset. This serves as the foundation for generating action trajectories in subsequent tasks.

Formally, given a dataset of trajectories $\mathcal{D} = \{\tau_i\}_{i=1}^N$, the diffusion model learns a conditional distribution $p_\theta(\tau|c)$, where c is the task specific context.

The training objective is as follows:

$$\mathcal{L}_{diffusion} = E[||\epsilon - \epsilon_\theta(\tau_t, t, c)||^2] \quad (1)$$

Where ϵ_θ predicts the noise in at timestep t . This allows us to generate actions which imitate the behavior specified in the dataset $\mathcal{D}$.

3.2 Generate Loss function Using LLM

To adapt the trained diffusion model to new tasks with specific constraints (e.g., execution speed or spatial boundaries), we utilize an LLM (For this project we used GPT-4o and Claude 3.5 Sonnet) to generate Python code that implements task-specific loss functions. The LLM takes natural language descriptions of user-defined constraints as input and outputs executable Python code for the corresponding loss function. The following is the prompt used to generate the loss functions.

System Prompt for LLM

[Template to fill in T,A depending on the task]

The function should be differentiable with respect to `x0_pred`, which contains actions with shape (B, T, A) , where T is the number of sequential steps in the trajectory and A contains deltas in x, y, z , and the end-effector. `robot_states` is of shape $(B, 1, 8)$ where the first 3 elements are the current position in x, y, z .

The output must be only Python code for the function, no imports but you have access to PyTorch, formatted like this:

```
def guidance_loss_fn(task_id, x0_pred, robot_state):
    # Implementation here
```

3.3 Loss-guided Trajectory Generation

To optimize trajectories generated by the diffusion model for user-defined tasks, we employ Loss-Guided Diffusion (LGD) [7], a plug-and-play framework that integrates task-specific loss functions into the reverse diffusion process. This method enables real-time trajectory adaptation without re-training the diffusion model by leveraging Monte Carlo-based approximations for efficient and accurate sampling.

During the diffusion process, LGD applies gradients to refine the trajectory depending on whether the model predicts noise (ϵ) or the fully denoised state (x_0):

1. **For noise-predicting models** (ϵ), guidance modifies the predicted noise (ϵ_θ) before denoising. The adjustment is made using a guiding gradient ($\nabla_\epsilon L$) as:

$$\epsilon' = \epsilon_\theta - \text{scale} \times \nabla_\epsilon L$$

Here, scale controls the magnitude of the adjustment, and ϵ' is used to recover a less noisy state, guiding the trajectory toward the desired behavior.

2. **For denoised-state-predicting models** (x_0), the gradients ($\nabla_x L$) directly update the noisy state x_t at each diffusion step:

$$x'_t = x_t + \text{scale} \times \nabla_x L$$

This method incrementally refines x_t , aligning the trajectory with user-defined tasks.

In both approaches, guidance can be applied iteratively n times per diffusion step, allowing for more precise adherence to constraints. This iterative process fine-tunes the trajectory while maintaining computational efficiency.

The full procedure is summarized in **Algorithm 1**, where LGD integrates the task-specific loss function from an LLM and applies guidance during the reverse diffusion process. By iteratively refining the noisy state across T diffusion steps, LGD produces optimized trajectories that meet user-defined requirements without requiring model retraining.

Algorithm 1 Diffusion Guidance with LLM-Generated Loss Function

Require: Noisy state x_T , diffusion model f_θ , LLM f_{LLM} , number of diffusion steps T , number of guidance steps n

- 1: Initialize $x \leftarrow x_T$
- 2: Obtain loss function L using f_{LLM} and user requirements
- 3: **while** not done **do** ▷ Iterate over diffusion steps
- 4: **for** $t = T, T - 1, \dots, 1$ **do**
- 5: Predict $\epsilon_\theta(x, t)$ or x_0 using model f_θ
- 6: **for** $i = 1$ to n **do** ▷ Apply guidance n times
- 7: **if** model predicts ϵ_θ **then**
- 8: Calculate $x_0 = \frac{x - \sqrt{1 - \alpha_t} \cdot \epsilon_\theta}{\sqrt{\alpha_t}}$
- 9: Calculate gradient $\nabla_{x_0} L$
- 10: Modify prediction: $\epsilon' = \epsilon_\theta - \text{scale} \times \nabla_{\epsilon} L$
- 11: **else** ▷ Model directly predicts x_0
- 12: Calculate gradient $\nabla_x L$
- 13: Modify prediction: $x = x + \text{scale} \times \nabla_x L$
- 14: **end if**
- 15: **end for**
- 16: Update x with the reverse diffusion process
- 17: **end for**
- 18: **return** Final denoised trajectory x_0 of horizon H
- 19: Execute m actions in the environment
- 20: **end while**

We plan to test our method on two key types of robotic constraints:

- **Temporal Constraints:** Tasks requiring specific execution durations are handled by penalizing deviations from desired timeframes using LLM-generated temporal loss functions.
- **Spatial Constraints:** Tasks involving precise positioning or motion within defined boundaries are managed using spatial loss functions that penalize positional errors.

4 Experimental Results

4.1 Setup

Contexts: We evaluate two distinct contexts in our experiments: (i) **Speed Constraint**, where tasks must be executed at user-defined speeds, and (ii) **Spatial Constraint**, where the robot must avoid entering a specified region or space during execution. For simplicity, we consider a scenario where the robot encounters a wall during execution— which was unseen during training.

Datasets: All experiments are conducted using the MetaWorld benchmark, which provides a diverse set of manipulation tasks designed to challenge learning algorithms. For the speed constraint context, we use the CLOSE DOOR task. For the spatial constraint context, we experiment on BUTTON-PRESS and PICK-N-PLACE tasks. At test time, we introduce walls as obstacles in these tasks, placing them between the robot’s end-effector and the target object. This tests the policy’s ability to adapt in zero-shot settings, as none of the training tasks include walls.

Training: We collect 100 demonstrations per task using scripted policies from the official MetaWorld codebase. For speed constraints, we vary the scripted policy speeds between 0.5 m/s and 1 m/s. At test time, we challenge the policy with a higher speed of 1.4 m/s and a lower speed for evaluation. For spatial constraints, we train on data collected using scripted policy for button press and pick-n-place tasks. For testing, we introduce a wall between the robot’s end-effector and the target object, specifying the wall’s pose and dimensions as the spatial region the policy must avoid.

Evaluation: Success rate is used as the primary metric, representing the percentage of successful rollouts or trials. We conduct 20 rollouts for speed constraint experiments and 40 rollouts per task for spatial constraint experiments. All results are averaged over 5 random seeds to ensure statistical reliability.

4.2 Results

We evaluate our proposed approach on 2 types of constraints - speed and spatial, on 3 tasks - close door, push button, pick and place, in the Metaworld environment.

Speed constraint: We prompt the policy to execute both faster and slower than speed in the original data distribution of the close door. With the loss-guided trajectory generation, our approach can execute the task at varying speeds as shown in the Fig. 3. We also show quantitative results of adapting to faster and slower speeds in Table 1. The speed loss generated from the LLM can be represented mathematically as:

$$\text{speed.loss} = \max(s - \text{max.speed}, 0) + \max(\text{min.speed} - s, 0), \quad (2)$$

where s is current speed

Speed	Success Rate	Overall Average Speed
unguided	1.0	1.635
slower	0.9	1.352
faster	1.0	1.643

Table 1: Zero-shot adapting to different speeds for Close Door task

Spatial constraint: At test-time, we roll out the policy by adding a wall in the scene (unseen during training). We observe that with our loss-guided generation, the policy is able to show improved performances on button press and pick-and-place task. The LLM-generated loss function for this constraint is shown in the Appendix.

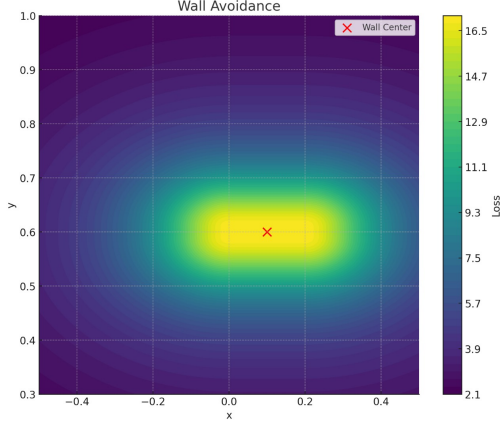


Figure 1: 2D visualization of wall avoidance-based loss

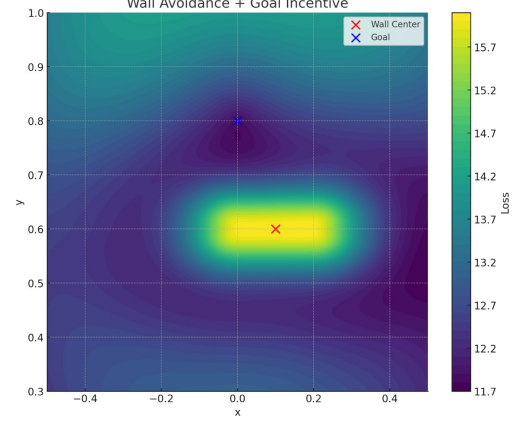


Figure 2: 2D visualization of wall avoidance and goal incentive-based loss

Task	Guidance	SR $\pm$ Std Dev
Button Press	No	1.00 $\pm$ 0.00
Button Press w/ Wall	No	0.53 $\pm$ 0.11
Button Press w/ Wall	Wall Penalization + Goal Incentive	0.63 $\pm$ 0.10
Button Press w/ Wall	Wall Penalization	0.26 $\pm$ 0.08
Pick and Place	No	0.80 $\pm$ 0.07
Pick and Place w/ Wall	No	0.40 $\pm$ 0.03
Pick and Place w/ Wall	Wall Penalization	0.80 $\pm$ 0.02

Table 2: Zero-shot Adaptation to Spatial constraints (wall)

For button press task, the original trained policy sees a drop of 47% when we place a wall between the robot gripper and button in the environment. However, when we introduce our loss-guided generation, the original policy improves task performance by 10% avoiding the unseen wall in the environment. For pick and place task, the original trained policy sees a drop of 40% when we place a wall between pick and place locations in the environment. With the loss-guided generation, we improve by 40% and can retain the same performance as in the original environment without wall.

For this constraint, we first experiment with wall avoidance-based loss (shown in Fig. 1) during roll out. This results in significant improvements of upto 40% in tasks like pick and place over without guidance. However, the policy does sometimes put over-importance to wall avoidance than executing the task successfully. Incorporating goal information in the loss function (shown in Fig. 2) does improve success rates by 10% in tasks such as button press. As shown in Fig. 2, the loss function introduces penalty for moving away from the goal which results in better task success rates.

Qualitative Results: Rollout videos for each of the constraints and tasks can be found [here](#).

5 Discussion and Analysis

In this project, we propose leveraging using diffusion model-based planners in conjunction with language models for zero-shot adapting to user contexts using loss-guided generation.

A key novelty of our approach is the ability to adapt to new scenarios using LLM-generated loss functions without any further retraining. This was evident using i.) the spatial constraint experiment

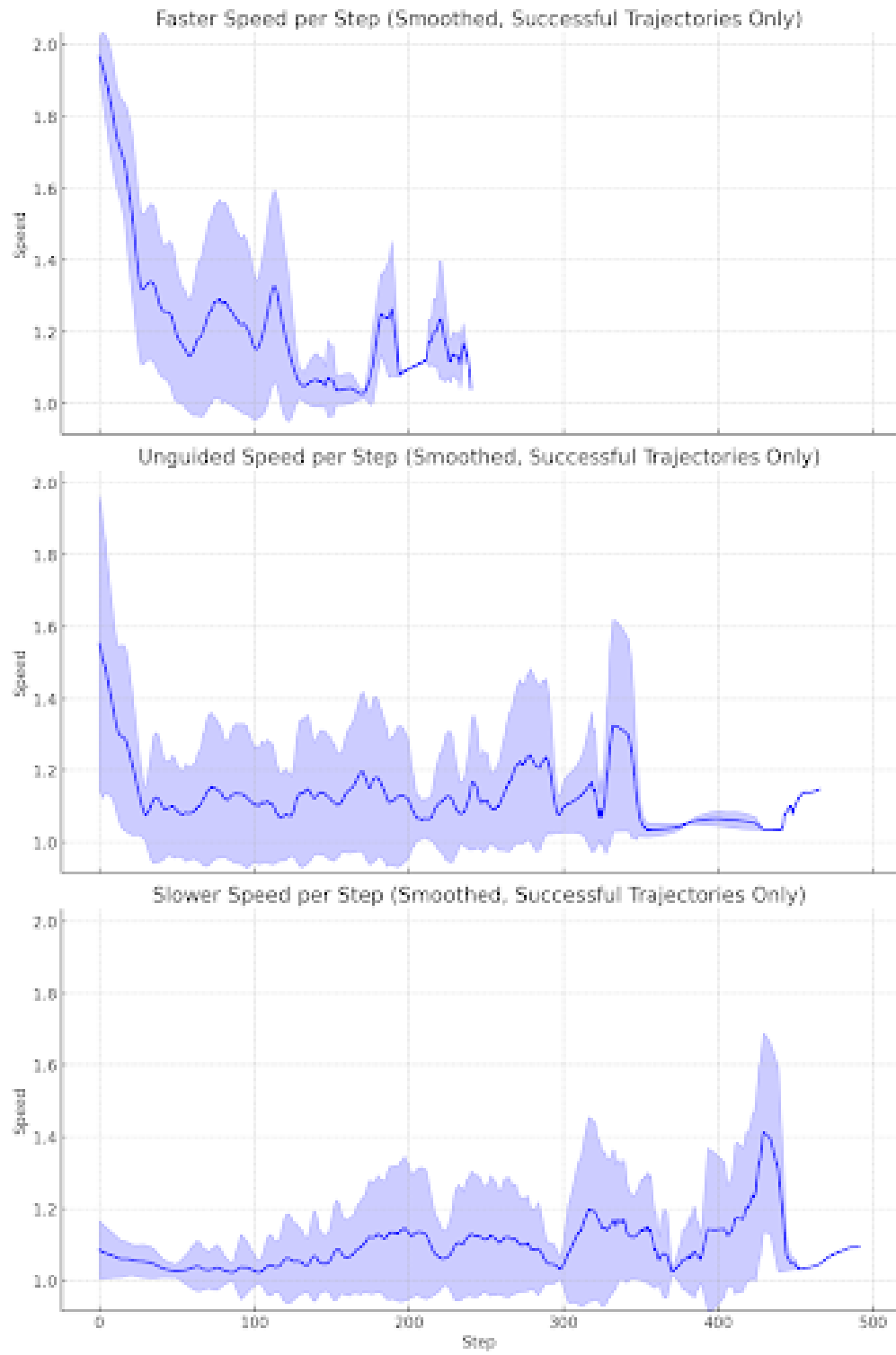


Figure 3: Close Door task policy adapting to varying speeds zero-shot during execution

where the policy adapted to avoiding walls placed during roll out (unseen during training). ii.) speed constraint - adapting to different speeds during policy execution.

However, there exist some key challenges in adapting this framework to wide range of contexts. The performance of the system depends on the proper adjustment of the gradient scaling factors, as poorly tuned parameters can reduce efficiency. Specifically, the policy needs to be carefully tuned to adapt between new user context and goal objective in a multi-loss setting for guided generation. Furthermore, the ability of the framework to generate diverse motion trajectories strongly depends on the initial training data distribution. For this, we experiment training our approach using multi-task datasets and adding Gaussian noise to trajectories during training. Further, the system also relies on good prompt engineering for loss function generation.

Future steps include devising techniques to perform precise tuning of multi-loss setting and gradient scaling, enriching training datasets and validating the approach in physical environments. Recent studies, such as "Scaling Diffusion Policy in Transformer to 1 Billion Parameters", explore the scalability of diffusion models in robotics, which could inform our efforts to enhance model robustness.

6 Conclusion

In this project, we set out to address the challenge of enabling robots to dynamically adapt their behaviors to user-defined contexts without retraining. By integrating diffusion-based trajectory planners with LLM-generated loss functions, we developed a system capable of zero-shot policy adaptation in real-time scenarios. Our approach successfully demonstrated adaptability to spatial and speed constraints, achieving zero-shot adaptation at test-time.

While challenges remain, particularly in fine-tuning gradient scaling factors, multi-loss optimization, and ensuring diversity in motion trajectories, our experiments underscore the potential of integrating diffusion models and LLMs for adaptive robotic control. Future work focused on enhancing dataset diversity, refining tuning mechanisms, and scaling the approach with advancements in diffusion model research could further bolster its applicability and robustness in real-world environments. Additionally, it would be intriguing to experiment with other types of constraints, such as temporal, dynamic obstacles, or energy constraints, to explore the full scope of the framework's adaptability.

References

- [1] M. Ahn, A. Brohan, N. Brown, Y. Chebotar, O. Cortes, B. David, C. Finn, C. Fu, K. Gopalakrishnan, K. Hausman, et al. Do as i can, not as i say: Grounding language in robotic affordances. *arXiv preprint arXiv:2204.01691*, 2022.
- [2] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, page 02783649241273668, 2023.
- [3] J. Ho, A. Jain, and P. Abbeel. Denoising diffusion probabilistic models. *Advances in neural information processing systems*, 33:6840–6851, 2020.
- [4] W. Huang, P. Abbeel, D. Pathak, and I. Mordatch. Language models as zero-shot planners: Extracting actionable knowledge for embodied agents. In *International conference on machine learning*, pages 9118–9147. PMLR, 2022.
- [5] M. Janner, Y. Du, J. B. Tenenbaum, and S. Levine. Planning with diffusion for flexible behavior synthesis. *arXiv preprint arXiv:2205.09991*, 2022.
- [6] Y. J. Ma, W. Liang, G. Wang, D.-A. Huang, O. Bastani, D. Jayaraman, Y. Zhu, L. Fan, and A. Anandkumar. Eureka: Human-level reward design via coding large language models. *arXiv preprint arXiv:2310.12931*, 2023.
- [7] J. Song, Q. Zhang, H. Yin, M. Mardani, M.-Y. Liu, J. Kautz, Y. Chen, and A. Vahdat. Loss-guided diffusion models for plug-and-play controllable generation. In A. Krause, E. Brunskill, K. Cho, B. Engelhardt, S. Sabato, and J. Scarlett, editors, *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 32483–32498. PMLR, 23–29 Jul 2023. URL <https://proceedings.mlr.press/v202/song23k.html>.